دفتر 09 / من الشرح للتطبيق

قيم افتراضية من غير مفاجآت

لو في قيمة بتستخدمها معظم الوقت، تقدر تخليها افتراضية. لكن لازم تفهم إمتى القيمة دي بتتكون، خصوصا لو كانت قائمة قابلة للتعديل.

في آخر الدفتر هتقدر تعمل إيه؟

تصمم دالة لها إعداد افتراضي، وتستخدم keyword arguments، وتتجنب مشاركة قائمة واحدة بين استدعاءات مختلفة.

هنمشي بالترتيب ده

1. نحدد المدخل الاختياري وقت التعريف.
2. نستخدم ترتيبا صحيحا للمدخلات.
3. نفهم مشكلة القائمة الافتراضية وحل None.

قبل ما تبدأ

تعريف الدوال وreturn من الدفتر 08. هنستخدم append كمثال لإضافة عنصر لقائمة، وتفاصيلها في الدفتر 11.

افتح ملف كود جنب الدفتر. اقرأ المثال، توقع النتيجة، وبعدها شغله. لو النتيجة مختلفة، ارجع للسطر اللي غيرها بدل ما تعيد نسخ الكود كله.

خريطة الدفتر: الفكرة ص 2 • تطبيق خطوة بخطوة ص 3 • الأخطاء ص 4 • تمارين ص 5 • حلول ص 6 • مراجعة ص 7

بايثون مع نوار

09

مدخلات مطلوبة واختيارية

شرح • أمثلة • تمارين • حلول

01 / استدعاء أبسط

اختار قيمة مناسبة لمعظم الحالات

PYTHON

```python
def greet(name, message="Hello"):
    return f"{message}, {name}"

print(greet("Nawar"))
print(greet("Nawar", message="Welcome"))
```

OUTPUT

```
Hello, Nawar
Welcome, Nawar
```

name مطلوب، وmessage اختياري لأن له قيمة افتراضية. لو لم تمرره، نستخدم Hello. لو مررته، القيمة الجديدة تخص هذا الاستدعاء. f قبل النص تسمح بإدخال القيم بين الأقواس المعقوفة.

ترتيب التعريف

في التعريف البسيط للدوال، المدخلات المطلوبة تأتي قبل المدخلات ذات القيم الافتراضية. وضع مدخل مطلوب بعد اختياري في هذا الشكل يسبب SyntaxError. خليك في الترتيب البسيط لحد ما تحتاج أنماطا متقدمة.

ترتيب الاستدعاء

في الاستدعاء البسيط، ضع القيم الموضعية أولا ثم القيم المسماة. استخدام الاسم مفيد لما القيمة رقم أو اختيار مش واضح المعنى. لا تملأ نفس المدخل موضعيا وبالاسم معا.

اختار الافتراضي بعناية

القيمة الافتراضية لازم تمثل سلوكا طبيعيا ومفهوما. لو وجودها يخفي معلومة أساسية، خلي المدخل مطلوبا.

02 / المشكلة والحل

القائمة الافتراضية بتعيش أطول

القيم الافتراضية تتقيم وقت تنفيذ تعريف الدالة، مش كل مرة نستدعيها. لو الافتراضي قائمة وعدلتها، التعديل يفضل موجودا في الاستدعاء التالي الذي يستخدم نفس الافتراضي.

PYTHON

```python
def add_item(item, items=[]):
    items.append(item)
    return items

print(add_item("A"))
print(add_item("B"))
```

OUTPUT

```
['A']
['A', 'B']
```

ده مثال لتوضيح الخطأ، مش نمط نستخدمه لقائمة مستقلة كل مرة. الحل: اجعل الافتراضي None، وأنشئ القائمة داخل الجسم عند الحاجة.

PYTHON

```python
def add_item(item, items=None):
    if items is None:
        items = []
    items.append(item)
    return items

print(add_item("A"))
print(add_item("B"))
```

OUTPUT

```
['A']
['B']
```

03 / ضبط التفاصيل

None مش معناها قائمة فاضية

ليه is None؟

None قيمة خاصة بتقول إننا لم نتلق قائمة هنا. فحص if not items هيمسك القائمة الفارغة أيضا، وقد يستبدل قائمة المستخدم وهو عايز يستخدمها. علشان نفرق، نفحص None تحديدا.

المستخدم مرر قائمة فعلا

في الحل السابق، لو مررت قائمة موجودة، الدالة ستعدل نفس القائمة. لو ده مش مقصود، أنشئ نسخة داخل الدالة قبل append. لازم سلوك التعديل يبقى واضحا في وصف الدالة.

الافتراضي مش بيتابع المتغير

PYTHON

```python
default_rate = 2
def scale(value, rate=default_rate):
    return value * rate

default_rate = 10
print(scale(3))
```

OUTPUT

```
6
```

القيمة 2 ارتبطت بالافتراضي وقت التعريف؛ تغيير default_rate بعد كده لم يغيرها. علشان معدل جديد يشتغل، مرره صراحة أو عرف السلوك المطلوب بوضوح.

فكرة الاختبار

لا تختبر الدالة مرة واحدة فقط. جرب استدعاءين متتاليين، وجرب تمرير القيمة صراحة وتركها افتراضية.

جرب قبل ما تبص على الحل

دورك إنت

تطبيق 01 / أس اختياري

اكتب power(base, exponent=2)، واختبر 3 بالافتراضي وبالأس 3.

تلميح صغير

استخدم ** واربط الاسم exponent بالقيمة.

تطبيق 02 / ليه النتيجة مشتركة؟

دالة تستخدم history=[] وتضيف عنصرا عند كل استدعاء. لماذا تراكمت العناصر؟

تلميح صغير

وقت إنشاء الافتراضي هو المفتاح.

تطبيق 03 / افتراضي آمن

اكتب make_list(value, items=None) وتأكد إن الاستدعاء بدون قائمة لا يحتفظ بقيم استدعاء سابق.

تلميح صغير

أنشئ القائمة داخل شرط is None.

قبل صفحة الحل: جرب حالة عادية وحالة على الحدود. اكتب النتيجة المتوقعة الأول، وبعدها قارنها باللي طلع فعلا.

مساحة لتجربتك

حلول مشروحة / قارن طريقة التفكير

نراجع اللي عملناه

حل 01 / أس اختياري

```python
def power(base, exponent=2):
    return base ** exponent

print(power(3), power(3, exponent=3))
```

النتيجتان 9 و27.

حل 02 / ليه النتيجة مشتركة؟

القائمة الافتراضية أنشئت وقت التعريف، وكل استدعاء بدون history استخدم نفس القائمة. استخدم None وأنشئ قائمة جديدة عند غياب المدخل.

حل 03 / افتراضي آمن

```python
def make_list(value, items=None):
    if items is None:
        items = []
    items.append(value)
    return items

print(make_list(5))
```

كل استدعاء افتراضي يبدأ بقائمة جديدة.

ورقة ترجع لها بسرعة

قبل ما تقفل الدفتر

راجع نفسك

- أفهم وقت تقييم الافتراضي.
- أستخدم None للقيم المتغيرة عند الحاجة.
- أختبر استدعاءات متتالية.

المصطلحات من غير تعقيد

Default : قيمة تستخدم عند غياب المدخل.

Keyword argument : تمرير قيمة باستخدام اسم المدخل.

Mutable : كائن يمكن تغييره في مكانه.

اختبار الفهم

هل كل قائمة داخل دالة مشكلة؟ لا؛ القائمة التي تنشأ داخل الجسم في كل استدعاء جديدة. المشكلة في مشاركة افتراضي قابل للتعديل دون قصد.

نكمل على إيه؟

الدفتر 10: lambda لدالة صغيرة من تعبير واحد، وإمتى def تكون أوضح.

لما تحتاج التفاصيل الرسمية

الشرح والأمثلة هنا مكتوبة للمبتدئ. الروابط التالية مراجع للمراجعة والتوسع، وليست نصا منقولا منها.

تعريف الدوال وتفاصيلها

https://docs.python.org/3/tutorial/controlflow.html